

Получите новый уровень в карьере

Повышайте свою квалификацию

Хабр



Чему научиться в этом году?



opensophy

1 час назад

mTLS: руководство — от теории к практике с управлением сертификатами и защиты сервисов

🔖 Простой 🕒 11 мин 👁 2.7K

Информационная безопасность*, Хранение данных*, Системное администрирование*, Управление проектами*, Серверно

Тutorial

Если вы когда-либо выставляли сервис в интернет и смотрели на логи — вы знаете, что происходит в первые минуты. Сканеры, боты, перебор паролей. Firewall помогает, но не всегда. VPN — хорошо, но не всегда удобно и сами протоколы в России к примеру хорошо работают. А что если сервер будет просто отказывать в соединении всем, у кого нет нужного криптографического сертификата — ещё до того, как они увидят страницу логина? Это и есть mTLS.

В статье разберём: что такое mTLS и как работает рукопожатие, как это связано с Zero Trust, от каких атак защищает и где принципиально бессилён, какие риски несёт сама PKI-инфраструктура и где чаще всего ошибаются при реализации. В конце — практика: как мы в Opensophy сделали `mtls.sh`, bash-скрипт для управления mTLS-сертификатами под Traefik, и почему архитектура «промежуточный CA на каждого клиента» позволяет мгновенно отзывать доступ без CRL и OCSP в Traefik.

Статья будет полезна всем, кто хочет защитить свои сервисы — будь то домашняя лаборатория, панели управления вроде Proxmox или Portainer/Dokploy, внутренние API или любой сервис, который не должен быть доступен всем подряд. Если коротко: если вы выставляете что-то в интернет и не хотите, чтобы туда мог зайти кто угодно — mTLS для этого и существует.

Что такое mTLS

Mutual TLS (mTLS) — расширение стандартного протокола TLS, при котором **обе стороны** соединения предъявляют и проверяют криптографические сертификаты. В обычном HTTPS только клиент проверяет сервер; сервер не знает, кто к нему подключается, и полагается на прикладную аутентификацию (логин, пароль, токен). В mTLS сервер дополнительно требует клиентский сертификат, подписанный доверенным центром сертификации (CA).

Если клиент не предъявляет сертификат или предъявляет невалидный — соединение разрывается **на транспортном уровне**, до любого HTTP-взаимодействия. Именно поэтому пользователь видит:

```
Доступ запрещён
Ваш сертификат отклонен сайтом или не был выдан.
ERR_BAD_SSL_CLIENT_AUTH
```

Объяснить с SourceCraft

Это знак того, что mTLS отработал штатно.

CA (Certificate Authority) — удостоверяющий центр. В случае mTLS организация выступает собственным CA: создаёт корневой сертификат и подписывает им клиентские сертификаты. В отличие от публичного TLS, где CA (Let's Encrypt, DigiCert) проверяет владение доменом, mTLS CA полностью под контролем администратора. (Вкратце это вы делаете сертификат и потом добавляете/делитесь с теми какое устройство должно получить доступ.)

Клиентский сертификат — X.509-сертификат, установленный на конкретном устройстве или в браузере. Содержит публичный ключ и метаданные (CN, Organization, срок действия). Подписан CA.

Приватный ключ — используется при TLS-рукопожатии для доказательства владения сертификатом. Никогда не передаётся серверу. На практике распространяется в составе `.p12` - файла (защищённый паролем контейнер, содержащий ключ + сертификат), который импортируется в браузер или на устройство — после чего ключ хранится в системном хранилище.

Как работает рукопожатие

Стандартный TLS (одностороннее доверие)

1. Клиент отправляет `ClientHello` — предлагает версии протокола и алгоритмы шифрования
2. Сервер отвечает `ServerHello` и отдаёт свой сертификат
3. Клиент проверяет сертификат сервера через доверенный CA
4. Стороны обмениваются `Finished` — канал установлен

Сервер при этом не знает, кто подключился. Аутентификация пользователя — задача приложения (логин, пароль, токен).

mTLS (взаимная аутентификация)

1. Клиент отправляет `ClientHello`
2. Сервер отвечает `ServerHello`, отдаёт свой сертификат и добавляет `CertificateRequest` — требование предъявить клиентский сертификат
3. Клиент отправляет свой сертификат
4. Клиент отправляет `CertificateVerify` — подпись приватным ключом, доказывающую что ключ действительно принадлежит ему
5. Сервер проверяет три вещи:
 - сертификат клиента подписан доверенным CA
 - клиент владеет соответствующим приватным ключом
 - сертификат не истёк и не отозван
6. Только после успешной проверки сервер отправляет `Finished`
7. Клиент отвечает `Finished` — зашифрованный канал установлен

Если на шаге 5 что-то не так — соединение разрывается. Клиент даже не увидит HTTP-ответа.

Что mTLS защищает — и что нет

mTLS работает на **транспортном уровне (L4/L5)**. Всё, что происходит до установления зашифрованного канала, — в зоне его ответственности.

Тип атаки	Как mTLS блокирует	Эффективность
Man-in-the-Middle (MitM)	Злоумышленник не владеет приватным ключом клиента и не может завершить TLS-рукопожатие	Высокая
Credential Stuffing	Без клиентского сертификата сервер не переходит к HTTP, форма авторизации недостижима	Полная
Brute Force / перебор паролей	Аналогично — нет доступа к HTTP без сертификата	Полная
Phishing (кража пароля)	Даже зная пароль, злоумышленник не может войти без сертификата жертвы	Высокая
Spoofing / Impersonation	Идентичность привязана к криптографическому сертификату, подделать который без CA невозможно	Высокая
Session Hijacking	Сессионный токен может быть привязан к mTLS-сертификату (certificate-bound tokens)	Средняя (зависит от реализации)
L7 DDoS от ботов без сертификата	Боты без сертификата отсекаются на уровне TLS-рукопожатия, не нагружая приложение	Высокая
Обнаружение сервиса (Shodan, Censys)	Сервер в режиме STRICT не возвращает баннеры неавторизованным клиентам; сканер видит ошибку TLS	Частичная

[Cloudflare](#) описывает использование mTLS (через API Shield) для защиты от неавторизованных запросов. Это документально подтверждённый сценарий применения, но не специфичная защита от volumetric DDoS.

После успешного TLS-рукопожатия mTLS передаёт управление прикладному уровню (L7). Всё, что происходит внутри установленного соединения — вне его ответственности. Согласно [OWASP Top 10](#) следующие атаки mTLS не блокирует:

Cross-Site Scripting (XSS). Если приложение уязвимо к XSS, вредоносный скрипт выполняется в браузере легитимного пользователя (у которого есть сертификат). Запросы скрипта к серверу будут успешно аутентифицированы через mTLS — он исходит от доверенного клиента.

SQL Injection. mTLS не анализирует содержимое передаваемых данных. Вредоносный SQL-запрос, отправленный авторизованным клиентом, будет доставлен до базы данных в полной сохранности.

Cross-Site Request Forgery (CSRF). Браузер жертвы уже имеет mTLS-сертификат и установленную сессию. Поддельный запрос, инициированный вредоносной страницей, будет принят сервером как легитимный.

Broken Access Control. mTLS аутентифицирует — подтверждает, *кто* подключился. Авторизация — *что ему разрешено* — остаётся задачей приложения. Если RBAC настроен некорректно, обычный пользователь с сертификатом может получить доступ к admin-интерфейсу.

Уязвимые зависимости (Log4Shell и подобные). Уязвимости в серверных библиотеках эксплуатируются через легитимный mTLS-канал авторизованным клиентом.

Инсайдерская угроза. Сотрудник с выданным корпоративным сертификатом имеет полный mTLS-доступ. Его вредоносные действия mTLS не ограничивает.

Все эти типы атак доступны тогда, когда атакующий имеет уже mTLS-сертификат.

Риски, ошибки реализации и инфраструктура

mTLS создаёт **асимметрию** в контексте отказоустойчивости. Запросы от клиентов без сертификата не доходят до веб-сервера — ресурсы CPU не тратятся на парсинг HTTP, рендеринг страниц, запросы к БД. Но каждое mTLS-рукопожатие требует больше вычислительных ресурсов, чем обычный TLS (проверка двух сертификатов, дополнительные криптографические операции). Злоумышленник может инициировать тысячи незавершённых рукопожатий, исчерпывая CPU сервера — это классическая DoS-атака на уровне TLS. От SYN-флуда и UDP-амплификации mTLS не защищает вовсе — для этого нужны внешние средства.

При одновременной перезагрузке большого количества подов в K8s инфраструктуре (например, после обновления кластера) все они одновременно обращаются к CA за новыми сертификатами. При высокой нагрузке CA может не справиться с запросами, и сервисы в режиме STRICT не смогут подняться — возникает каскадный отказ. Меры профилактики: кэширование сертификатов, горизонтальное масштабирование CA, промежуточные центры выдачи, jitter при обновлении сертификатов.

Компрометация приватного ключа клиента. Если злоумышленник похитит `.p12`-файл с устройства пользователя (вместе с паролем или без), он получит полноценный mTLS-доступ до момента отзыва сертификата. Меры защиты: хранение ключей в аппаратных токенах (YubiKey, TPM), защита устройств паролём/биометрией, мониторинг аномальных сессий. Для большинства одиночных homelab это вообще не требуется, нужно только если у вас доступ уже от 2-5 человек к сервисам.

Компрометация корневого CA. Это самый опасный сценарий. Если приватный ключ корневого CA украден — злоумышленник может выпустить любой клиентский сертификат и получить доступ ко всему, что защищено mTLS. Вся инфраструктура доверия рухнет. Рекомендуемые меры:

- **Держите CA-ключ offline.** Идеально — на отдельной машине без доступа в сеть, которая включается только для выпуска новых сертификатов.
- **HSM (Hardware Security Module)** — специализированное железо для хранения криптографических ключей. Грубо говоря, это может быть флешка, из которой ключ невозможно вытащить — устройство только подписывает данные внутри себя, наружу ключ не отдаёт никогда. Для homelab это избыточно, но в продакшне где требуется максимум безопасности это стандарт. (Хотя в других случаях и предприятиях флешки запрещены.)
- **Ограничьте доступ к файлу ключа** — права 600, только root, никаких резервных копий в облаке.

Проблемы отзыва. Если говорим про K8S: в Service Mesh (к примеру Istio) проверка CRL и OCSP часто отключается из соображений производительности. Это означает, что скомпрометированный сертификат будет работать до истечения срока его действия. Отраслевая практика — использование **короткоживущих сертификатов** (от 2 до 24 часов) вместо классических механизмов отзыва. Istio по умолчанию выдаёт сертификаты сроком 24 часа. Если сервер настроен на обязательную проверку OCSP в режиме «fail-closed», атака на OCSP-сервер делает невозможным доступ для всех клиентов — даже легитимных.

Ошибки реализации. Исследователь [GitHub Security Lab](#) Майкл Степанкин представил на Black Hat USA 2023 и DEF CON 31 детальный разбор уязвимостей в реализациях mTLS. Ключевые выводы:

Распространённая архитектура: фронтенд-прокси (Nginx, HAProxy, Traefik) выполняет mTLS-аутентификацию и передаёт данные о клиентском сертификате бэкенду через HTTP-заголовки (например, `X-Client-Cert` или `X-SSL-Client-DN`). Если бэкенд не проверяет, что заголовки пришли именно от доверенного прокси, злоумышленник может отправить поддельные заголовки и выдать себя за другого пользователя.

Сертификат с `CA:TRUE` в Basic Constraints может быть использован для подписи других сертификатов. Если сервер не проверяет это поле, клиент с CA-сертификатом может выпустить произвольный клиентский сертификат и аутентифицироваться от имени любого пользователя. Степанкин обнаружил подобные CVE в нескольких open-source identity servers.

Если сервер выполняет проверку OCSP или CRL до верификации подписи сертификата, злоумышленник может использовать URL из расширения CRLDP для SSRF-атак или JNDI-инъекций (как в случае с Apero CAS). **Правило:** отзыв проверяется **после** валидации подписи, не до.

Примечание если проверять источник: SAP-кейс описывался как «случай с Black Hat», однако в публичных материалах Black Hat USA 2023 (Степанкин) описаны уязвимости в open-source серверах (Apero CAS и других), а не в SAP-продуктах. Реальные уязвимости Степанкина — в identity servers с неправильной обработкой цепочек сертификатов.

Практика: mtlsh

Поскольку Istio заточен в основном под k8s, а другие решения для меня показались довольно «большими» — в плане того чтобы выпускать сертификаты приходится писать много команд, поэтому представляю вам [скрипт который упрощает выпуск сертификатов и делает управление намного легче](#). Скрипт заточен в основном под docker/podman итп контейнеры где стоит **Traefik**.

Хотелось бы предложить it-сообществу решение проблемы в Traefik а именно то что Traefik не поддерживает CRL. Это означает что отзыв сертификата не работает, скрипт решает эту проблему следующим путём.

Ключевое решение в [mtls.sh](#) — создание **отдельного промежуточного CA для каждого клиентского сертификата**:

```
Корневой CA
├─ Int-CA (alice)  -> client.crt (alice)
├─ Int-CA (bob)   -> client.crt (bob)
└─ Int-CA (carol) -> client.crt (carol)
```

[Объяснить с](#) 

Все промежуточные CA собираются в один файл `clients-bundle.crt`, который передаётся Traefik. При отзыве сертификата alice её промежуточный CA просто исключается из bundle. Bob и Carol не затронуты. Traefik подхватывает обновлённый bundle без перезагрузки (через file provider). Никакого CRL, никакого OCSP, никаких задержек. Этот подход позволяет реализовать отзыв без стандартных механизмов отзыва сертификатов — что особенно ценно в homelab-окружениях, где настраивать OCSP-ответчик может быть нецелесообразно.

Скрипт работает в двух режимах интеграции с Traefik. Режим **new** создаёт новый домен и полностью управляет его конфигурацией:

```
Домен: myapp.example.com
Target: http://localhost:3000
-> Создаёт роутер, сервис и TLS-опции в mtls-manager.yml
```

[Объяснить с !\[\]\(5eb1325dfdc3f1cad8426726c0db51cd_img.jpg\)](#)

Режим **patch** добавляет mTLS к существующему роутеру (например, созданному Dokploy) — минимально инвазивно:

```
Существующий файл: /etc/dokploy/traefik/dynamic/dokploy.yml
Роутер: my-existing-app
-> Добавляет строку options: mtls-my-existing-app
```

[Объяснить с !\[\]\(950a62bbddad88d64435fd35607dfc42_img.jpg\)](#)

Пресеты к путям также есть для Dokploy т.к это основная платформа для управления контейнерами и сайтами в opensource проектах. [Вы можете создать запрос в гитхабе](#) чтобы добавить новые пресеты или сделать самому их.

Жизненный цикл сертификата в [mtls.sh](#):

1. Добавить сервис (new или patch)
2. Создать сертификат -> получить client.p12
3. Импортировать .p12 на устройство
4. Доступ к сервису работает
5. При необходимости -> Отозвать (почти мгновенная блокировка)
6. При необходимости -> Удалить файлы

[mtls.sh](#) — инструмент для homelab и небольших инфраструктур. Он **не был создан** для продакшн-кластеров с тысячами сервисов, ECDSA-сертификатов (только RSA) и полноценного PKI с CRL/OCSP.

Возможные проблемы

Очистка браузерных хранилищ. При импорте `.p12` браузер сохраняет не только клиентский сертификат, но и промежуточный CA из цепочки. При перевыпуске сертификатов старые промежуточные CA накапливаются в хранилище. Примеры ниже помогут убрать «хлам» из настроек браузера.

Linux:

```
# Найти базу сертификатов
find /home -name "cert9.db" 2>/dev/null

# Установить certutil
apt install libnss3-tools -y
```

```
# Просмотр (Chrome или другого браузера)
certutil -L -d /home/<user>/.local/share/pki/nssdb/

# Удаление (по точному имени из колонки Certificate Nickname)
certutil -D -d /home/<user>/.local/share/pki/nssdb/ -n "opensophy - mTLS-Manager"
```

[Объяснить с](#) 

Не удаляйте сертификаты с атрибутом `u,u,u` — это активные клиентские сертификаты.
Промежуточные CA имеют пустой атрибут `,, .`

Windows:

```
# Просмотр
Get-ChildItem -Path Cert:\CurrentUser\CA | Where-Object { $_.Subject -like "*opensophy"

# Удаление
Get-ChildItem -Path Cert:\CurrentUser\CA | Where-Object { $_.Subject -like "*opensophy"
```

[Объяснить с](#) 

Firefox на Windows использует собственное хранилище: `about:preferences#privacy` -> Просмотр сертификатов -> Центры сертификации.

Android (в зависимости от ОС/версии/модели): Настройки -> Безопасность -> Шифрование и учётные данные -> Доверенные данные -> вкладка Пользователь. Или: Настройки -> в поиске вводим Сертификат и ищем подобные данные в устройстве.

iOS / iPadOS (в зависимости от ОС/версии/модели): Если сертификат установлен через профиль: Настройки -> Основные -> VPN и управление устройством -> найти профиль -> Удалить.

Выводы и сравнение

mTLS — это отличный инструмент для конкретной задачи: убедиться, что к вашему сервису подключается только то устройство, которому вы явно выдали сертификат. Для homelab это означает что Proxmox, Portainer, Home Assistant и прочие админки перестают быть видны для сканеров и ботов — они просто получают ошибку TLS ещё до того, как увидят страницу логина. Никакого брутфорса, никакого credential stuffing, никаких попыток эксплуатировать форму входа. При этом важно понимать границы: mTLS доверяет устройству, а не человеку за ним. Если устройство скомпрометировано — скомпрометирован и доступ. Если приложение уязвимо к XSS или SQL-инъекции — mTLS это не исправит. Он работает на транспортном уровне и выше не смотрит.

`mtls.sh` мы написали потому что хотели простое решение без лишних зависимостей и написания огромного кол-во команд — только `openssl` и `python3`. Архитектура с промежуточным CA на каждого клиента решает главную проблему Traefik: отсутствие поддержки CRL. Отозвать доступ — значит просто убрать один сертификат из bundle. Никаких перезапусков, никаких сложных конфигураций.

Если вы только начинаете — попробуйте на одном сервисе. Защитите что-то одно, посмотрите как это работает, потом масштабируйте. mTLS проще в эксплуатации, чем кажется на первый взгляд.

Критерий	mTLS	VPN	IAP
Уровень OSI	L4 (транспортный)	L3 (сетевой)	L7 (прикладной)
Что защищает	Конкретный сервис/ домен	Весь сетевой стек	Конкретное приложение
Установка клиента	Импорт .p12 в браузер	VPN-клиент	Нет (браузер + SSO)
Удобство для пользователя	Прозрачно после импорта	Нужно включать VPN	Удобно (SSO/2FA)
Страница логина в интернете	Нет	Нет	Да
Подходит для IoT/API	Отлично	Сложно	Не применимо
Управление доступом	По устройству	По IP/ пользователю	По идентификатору пользователя

Когда я выбрал бы mTLS: нужен доступ только к конкретному сервису с устройства, где невозможно или нежелательно устанавливать VPN-клиент или VPN-протоколы/клиент попросту не работает; нужна защита API или автоматизации без UI-аутентификации.

Когда я выбрал бы VPN: нужен полный доступ к инфраструктуре, удалённая работа.

Когда я выбрал бы IAP: публичный сервис с множеством пользователей, SSO и 2FA обязательны, не нужно управлять сертификатами.

Статья доступна на сайте [opensophy](#). Статьи, блоги и документации в первую очередь выходят и обновляются на сайте с указанием автора и соавторов.

Теги: [mtls](#), [zero trust](#), [traefik](#), [devsecops](#), [сети](#), [защита данных](#), [защита информации](#), [защита от ddos](#), [защита сайта](#), [кибербезопасность](#)

Хабы: [Информационная безопасность](#), [Хранение данных](#), [Системное администрирование](#), [Управление проектами](#), [Серверное администрирование](#)



Редакторский дайджест

Присылаем лучшие статьи раз в месяц

Подписаться

Оставляя почту, я принимаю [Политику конфиденциальности](#) и даю согласие на получение рассылок



32K+

Охват за 30 дней

15

Карма

@opensophy

Пользователь

Подписаться



Поток Информационная безопасность доступен 24/7 благодаря поддержке друзей Хабра

Хабр Курсы для всех

РЕКЛАМА

Практикум, Хекслет, SkyPro, авторские курсы — собрали всех и попросили скидки. Осталось выбрать!

Перейти

Перейти в поток Информационная безопасность

Комментарии 2

Публикации

ЛУЧШИЕ ЗА СУТКИ

ПОХОЖИЕ



the_bat

23 часа назад

Ремонт блока питания с Power Delivery. 470 граммов электроники

Простой

4 мин

16K

+96

30

27



interpres

18 часов назад

Электроинструмент становится хуже, и это делается намеренно

Простой

11 мин

26K

Обзор

Перевод

+70

45

82



prohronus

18 часов назад

Как ИИ-агенты стали новым оружием скамеров на Хабр Карьере

5 мин

9.2K

Кейс

+51

9

9



alizar

22 часа назад

Готовимся к отключению. Эффективные форматы для упаковки и раздачи HTML-страниц

Простой

7 мин

12K

Обзор

+37

59

18



beget_com

23 часа назад

Механический калькулятор. Как работает арифмометр?

Средний

10 мин

9.5K

Обзор

+37

23

8



TrexSelectel

19 часов назад

Linux 7.0 и что изменилось в ядре после очередного цикла разработки

5 мин

8.7K

Обзор

+27

10

0



AlexSerbul

23 часа назад

Программирование с AI-ассистентом — похороните меня под плинтусом

Средний

14 мин

9.9K

Мнение

+26

33

11



mariklolik

18 часов назад

Как переложить нагрузку по code review с разработчиков на LLM

Средний

7 мин

7.4K

Кейс

+23

23

6



nick_dyuba

22 часа назад

Легенды 90-х — кто придумал и производил жвачки Turbo, Love is... и ТіріТір

5 мин

7K

Recovery Mode

+22

8

5



Mimizavr

23 часа назад

«Великое очищение» в работе с контентом: что осталось от роли редактора

🕒 11 мин 👁 6.9K

📦 +17

🔖 9

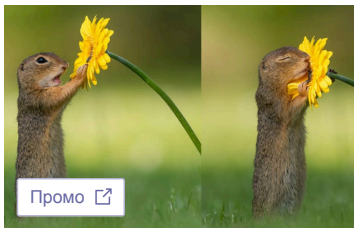
💬 7

Тысяча нюансов: как управлять производством лекарств

Турбо

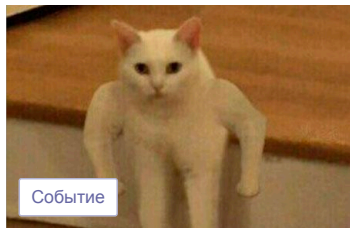
Показать еще

МИНУТОЧКУ ВНИМАНИЯ



Промо

Запахло весной и скидками в Промокодусе



Событие

Качаем к лету хард- и софт-скиллы на айтишных ивентах



Турбо

Удалёнка в фарме: большие данные и никакой магии

ВОПРОСЫ И ОТВЕТЫ

Как настроить DMZ, проброс портов и NAT loopback?

Компьютерные сети · Средний · 1 ответ

Проблема с получением IP по DHCP: веб-сервер на VMWare Player и роутер с локалкой?

Компьютерные сети · Средний · 3 ответа

Как клиентов роутера кинетик прокинуть в туннель WG?

VPN · Простой · 1 ответ

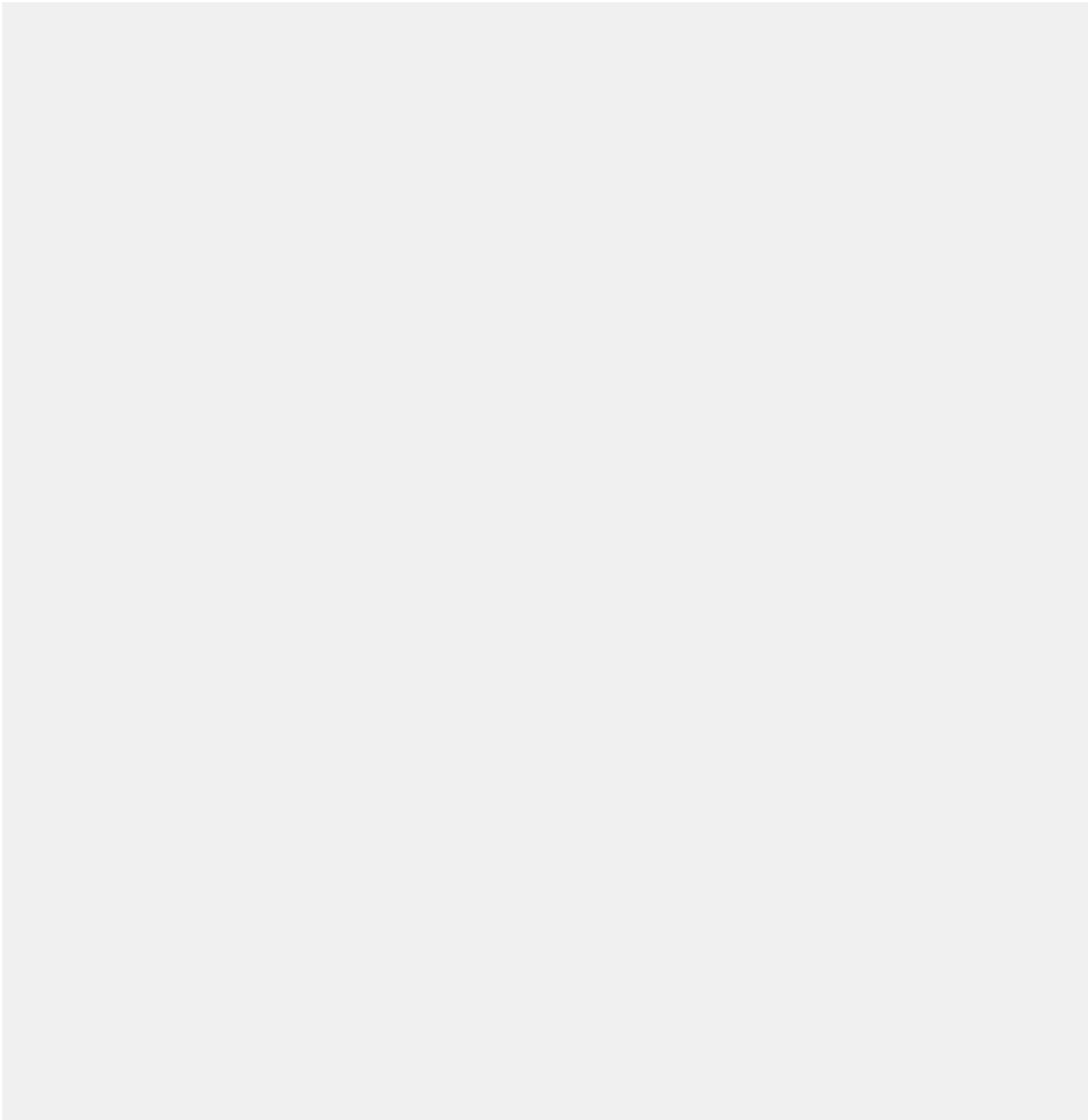
Почему отваливается домен apple.com?

Компьютерные сети · Простой · 2 ответа

Почему появляется ошибка при подключении к базе данных?

Linux · Простой · 3 ответа

[Больше вопросов на Хабр Q&A](#)



ЧИТАЮТ СЕЙЧАС

Электроинструмент становится хуже, и это делается намеренно

 26K  82 

Тим Кук покидает пост Apple. Новый глава — «отец» Apple Silicon Джон Тернус

 13K  14 

Представлен проект KillerPDF — редактор PDF с открытым исходным кодом для Windows 10/11

 6.7K  15

Пользователь купил Atomic Heart на VK Play, но не смог запустить игру из-за сетевых проблем

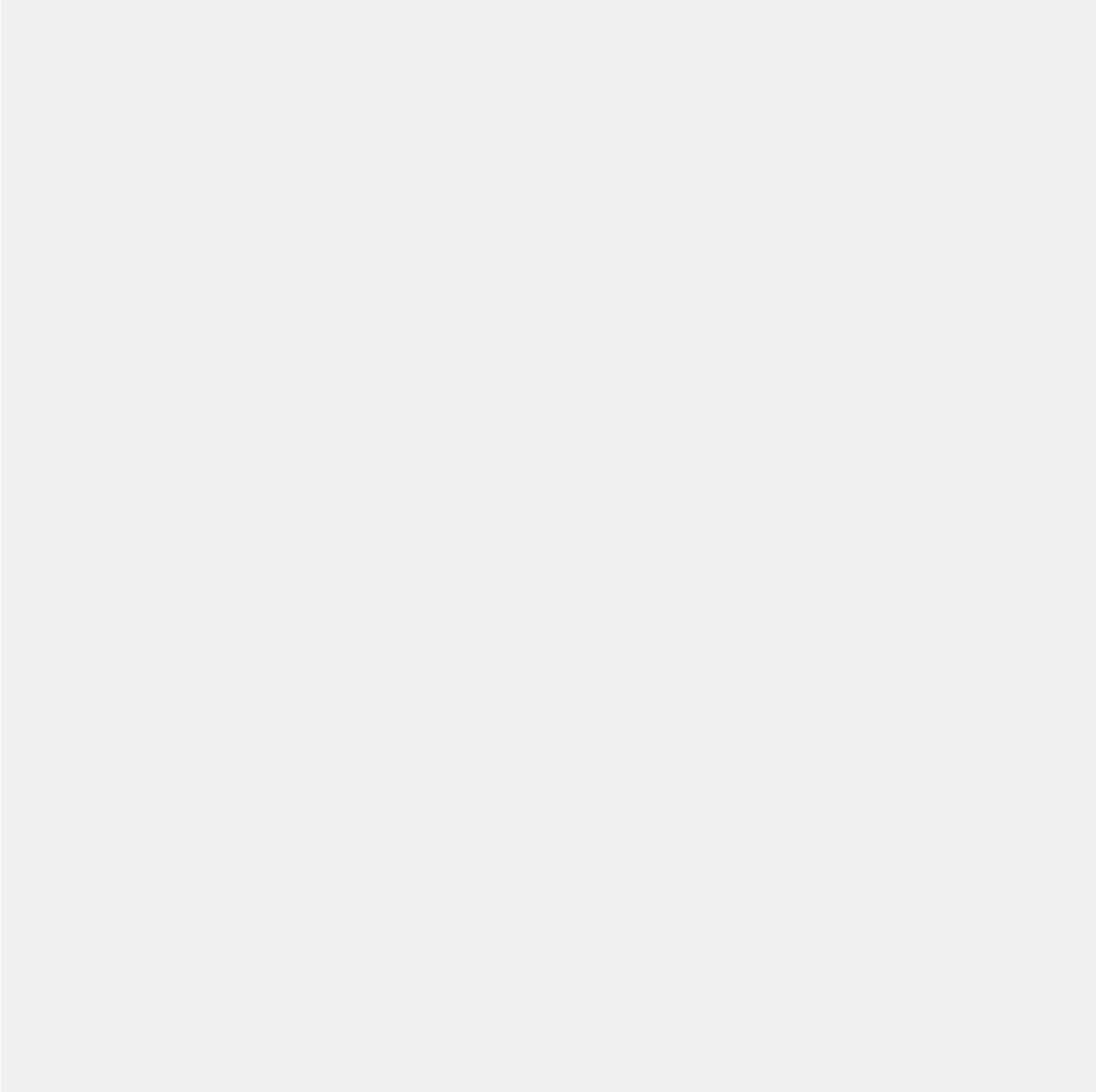
 5.3K  11

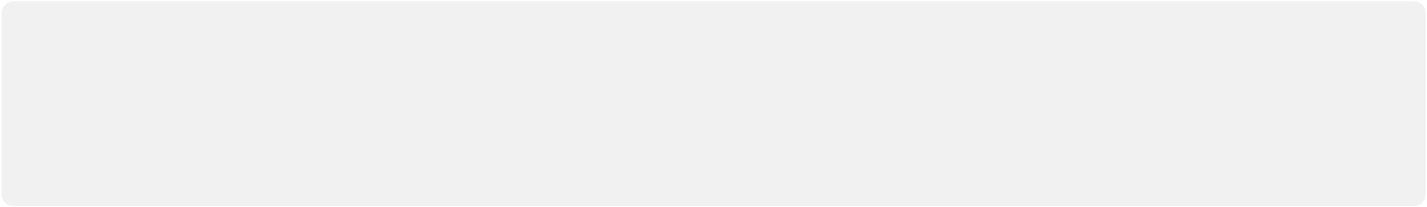
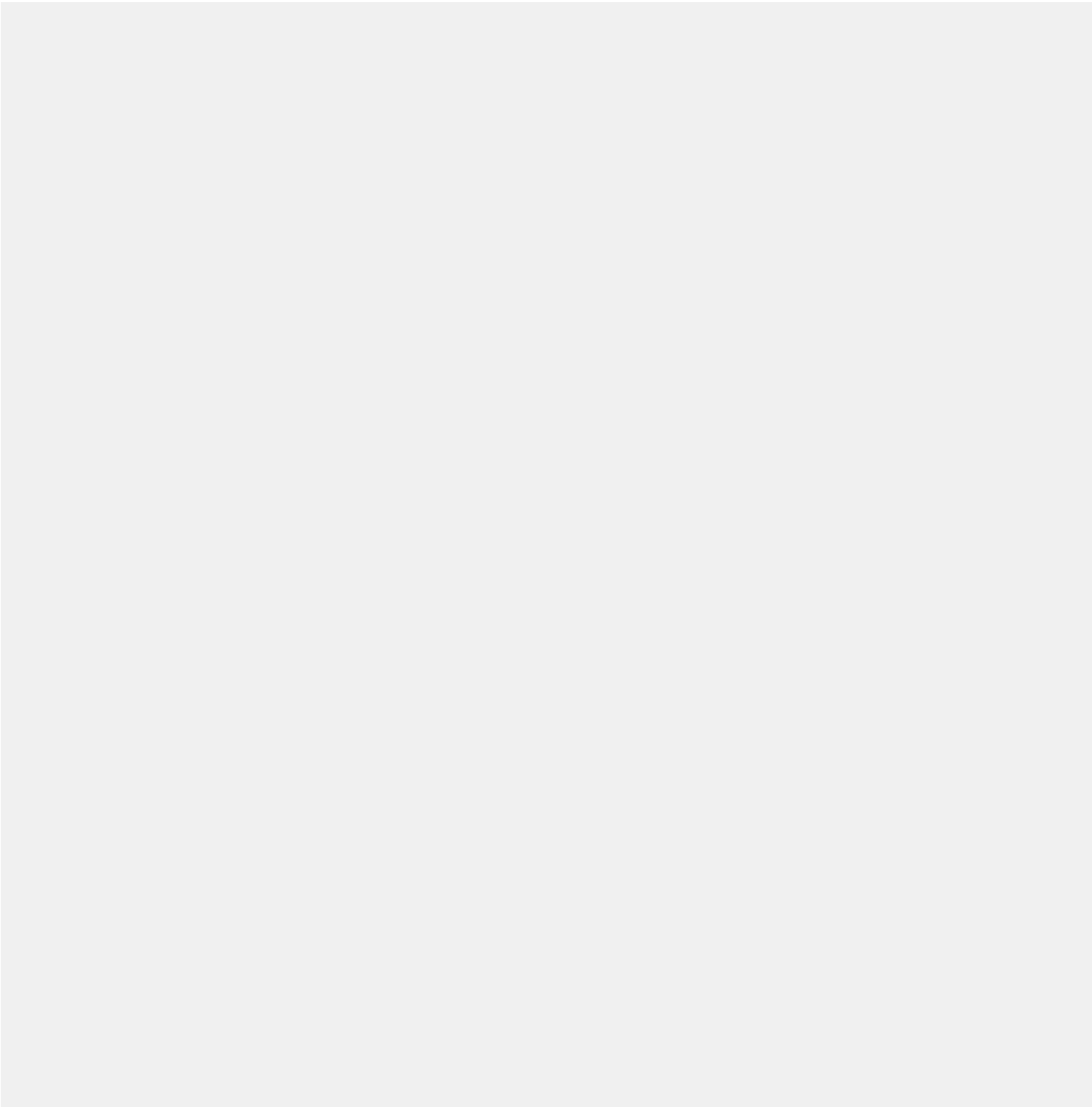
+185% за 13 часов: как Kimi K2.6 переписала 8-летний движок

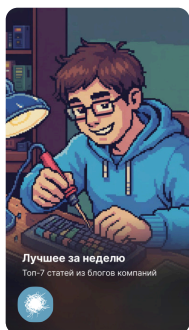
 14K  9

Тысяча нюансов: как управлять производством лекарств

Турбо







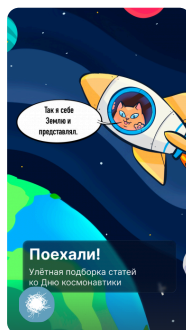
Годнота из блогов компаний



Только 10% решат эту головоломку



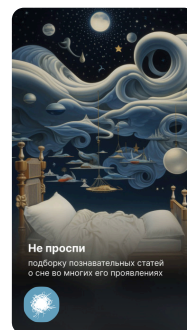
Хочу 450К



Вперёд к звёздам

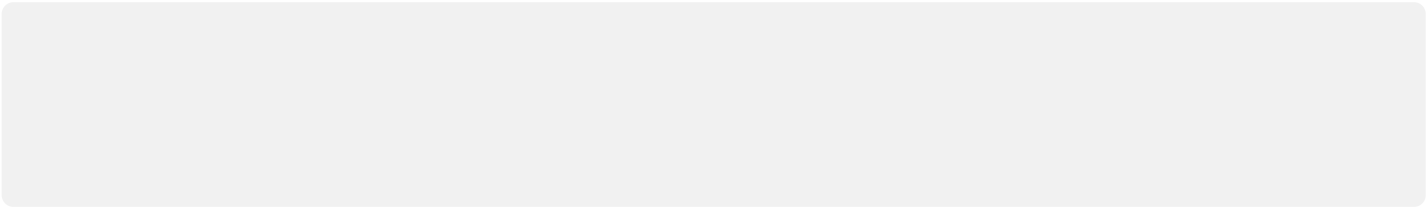
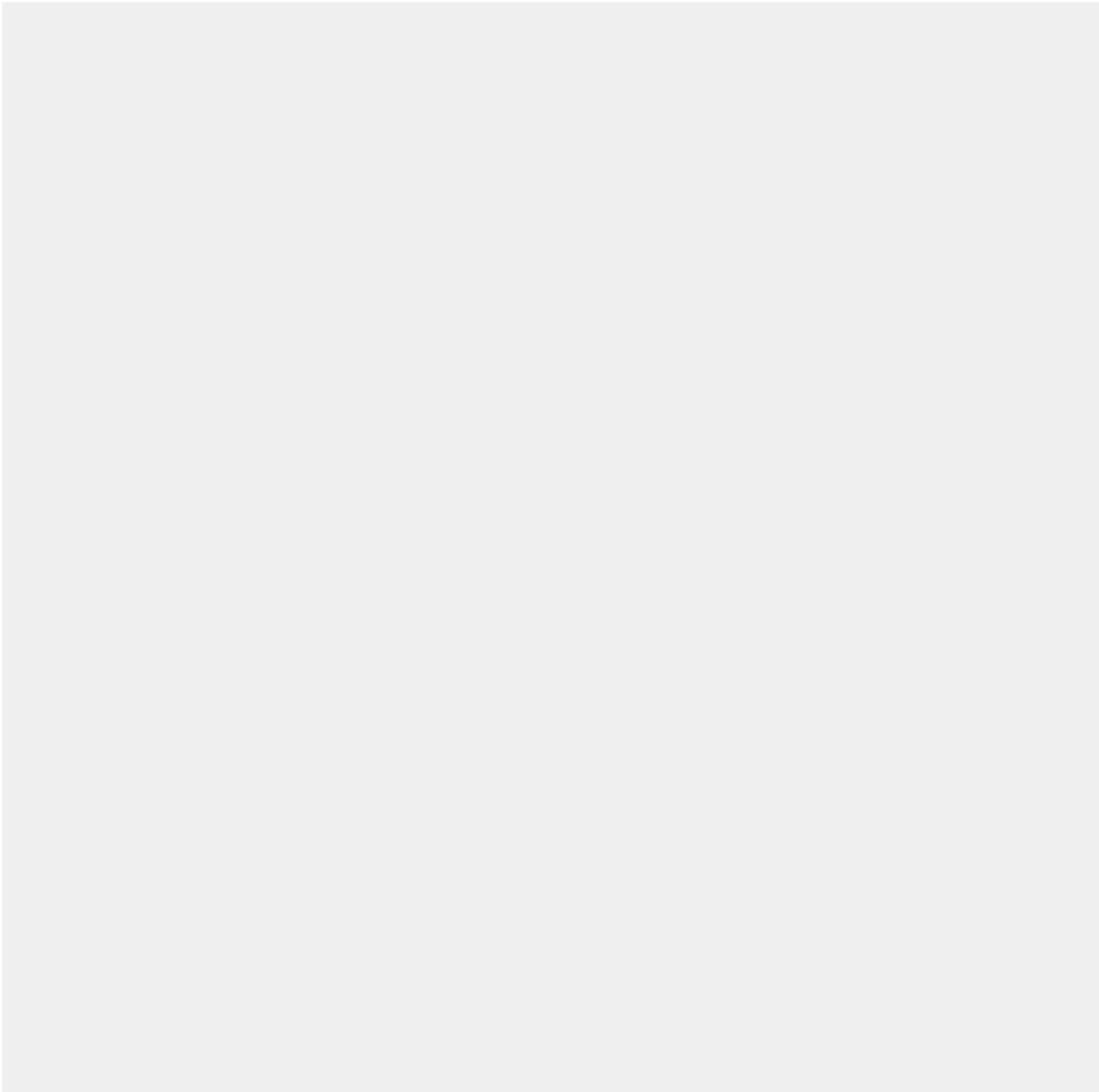


Там на неведомых дорожках...



Самое время поспать





Gray horizontal bar

Gray horizontal bar

Gray horizontal bar



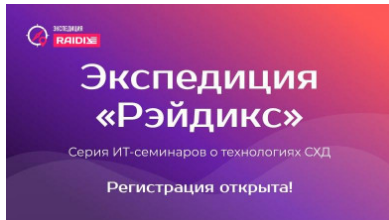
2 марта – 10 августа

PROBA Awards — международная коммуникационная премия, учрежденная в 2000 году

Санкт-Петербург

Маркетинг

[Больше событий в календаре](#)



3 марта – 9 июня

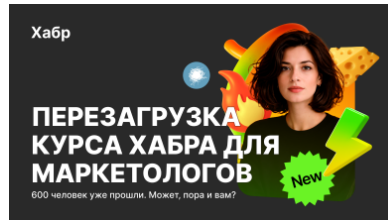
Экспедиция «Рэйдикс»

Екатеринбург • Новосибирск •
Краснодар • Минск • Калининград •
Ростов-на-Дону

Разработка

Другое

[Больше событий в календаре](#)



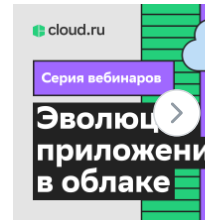
9 марта – 1 июня

Всё, что нужно, чтобы стартовать продвижение бренда на Хабре

Онлайн

Маркетинг

[Больше событий в календаре](#)



19 марта – 28 апреля

Серия вебинаров «Эволюция приложений в облаке»

Онлайн

Разработка

[Больше событий в календаре](#)

Ваш аккаунт

Войти
Регистрация

Разделы

Статьи
Новости
Хабы
Компании
Авторы
Песочница

Информация

Устройство сайта
Для авторов
Для компаний
Документы
Соглашение
Конфиденциальность

Услуги

Корпоративный блог
Медийная реклама
Нативные проекты
Образовательные программы
Стартапам



Настройка языка

Техническая поддержка

© 2006–2026, Habr